

St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2025-2026

Class: BE-ITA/B, Semester: VIII

Subject: BlockChain Lab

Experiment – 5

1. **Aim:** To implement smart contract using truffle framework

2. **Objective:** To ...

- explore working of truffle framework.
- explore working of Ganache local ethernet network.
- explore deployment scripts.

3. **Lab outcome:** After performing the experiment, the students will be able to **implement** smart contracts in Ethereum using different development frameworks (PO3, PSO2, BL3)

4. **Prerequisite:**

- Fundamental knowledge of blockchain
- Knowledge of the Ethereum platform
- Familiarity with the Solidity programming language and JavaScript

5. **Requirements:** The following are the requirements –
Truffle Framework, Ganache Provider, Visual Studio Code etc.

6. **Pre-Experiment Theory:**

What is Truffle Framework?

We have various setup options for deploying, migrating, and accessing smart contracts. Depending on the level of control and visibility we want into the EVM (Ethereum Virtual Machine), we can choose from using an online IDE like Remix, to running a full Ethereum mining node via Geth. Truffle is a world-class development environment, testing framework and asset pipeline for blockchains using the EVM. Truffle is widely considered the most popular tool for blockchain application development.

Some of the features of Truffle suite are

- Built-in smart contract compilation, linking, deployment and binary management.
- Automated contract testing for rapid development.
- Scriptable, extensible deployment & migrations framework.
- Network management for deploying to any number of public & private networks.
- Package management with EthPM & NPM, using the ERC190 standard.
- Interactive console for direct contract communication.
- Configurable build pipeline with support for tight integration.
- External script runner that executes scripts within a Truffle environment.

7. **Laboratory Exercise**

A. Steps to be implemented.

To Follow the procedure given below to build smart contract in Remix IDE

1. Download and install Nodejs from command-prompt terminal, if it is not installed.
2. Create a new directory using command mkdir, with name Ethereum (or any other name).
3. Change directory to new directory using cd Ethereum
4. Set up truffle using command npm install -g truffle

5. Open folder/directory Ethereum using Visual Studio, observe empty folder

From terminal type command, truffle init and observe the directory Ethereum with new subfolders

6. In Visual Studio, under contract folder create a new file HelloWorld.sol and write a smart contract (as done in exp-1).
7. Under migration folder create a new js file with name 2_helloWorld_migration.js and paste the following js code in the file

```
var HelloWorld = artifacts.require("./HelloWorld.sol");
module.exports = function(deployer) {
  deployer.deploy(HelloWorld);
};
```

8. Open truffle-config.js, delete all its contents and paste the following code
module.exports = {
 networks: {
 development: {
 // from: "", // Defaults to first address from Ganache host:
 "127.0.0.1",
 port: 7545,
 network_id: "*"
 }
 }
};
9. Now from terminal run command truffle compile
10. Open Ganache provider and from terminal run command truffle migrate
11. Now to see the output of smart contract run command truffle console

G. Program Code

1. Write a smart contract HelloWorld.sol and deploy it using Ganache and Truffle.
2. Write a program DonateEther.sol, create uint public variable 'balance' and initialize it to 0 using constructor() which is public, write a function contribute(), make it public and payable. Inside the function body write balance+=msg.Value; then create migration.js file for this contract as done in HelloWorld program. Deploy the contract on Ganache local Ethereum network using truffle.

8. Post Experimental Exercise-

H. Questions:

1. List down the details of components of Truffle suite.
2. Deploy all smart contracts studied in expt-1 and expt-2 using Truffle and Ganache.

I. Results/Observations/Program output:

Present the program input/output results if any and comment on the same.

J. Conclusion:

1. Write what was performed in the experiment.
2. Write which tools you used to perform the experiment
3. Write what you inferred from the output obtained.

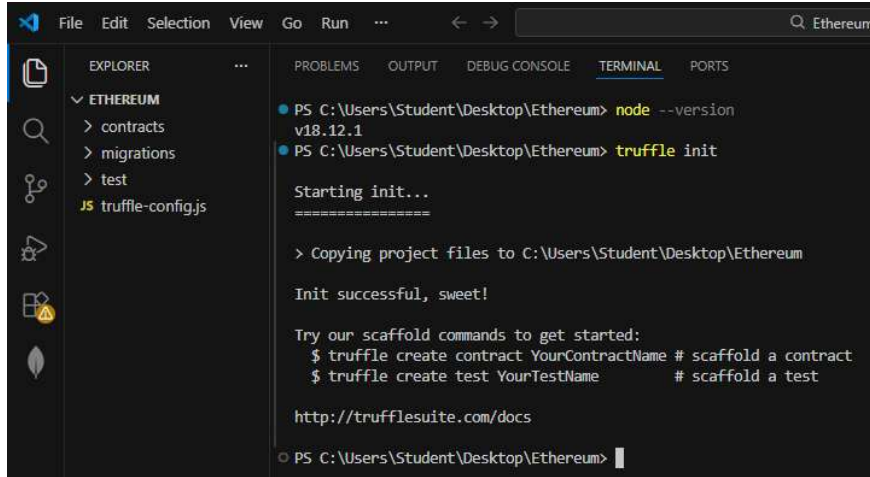
K. References:

[1] <https://trufflesuite.com/docs/truffle/quickstart/>

[2] Mastering Ethereum, Building Smart Contract and Dapps, Andreas M. Antonopoulos

OUTPUTS:

1. Initializing the truffle project using truffle init command in the desired folder.



The screenshot shows the VS Code interface with the Explorer pane on the left showing the 'ETHEREUM' folder containing 'contracts', 'migrations', 'test', and 'truffle-config.js'. The Terminal pane on the right shows the following commands and output:

```
PS C:\Users\Student\Desktop\Ethereum> node --version
v18.12.1
PS C:\Users\Student\Desktop\Ethereum> truffle init

Starting init...
=====
> Copying project files to C:\Users\Student\Desktop\Ethereum

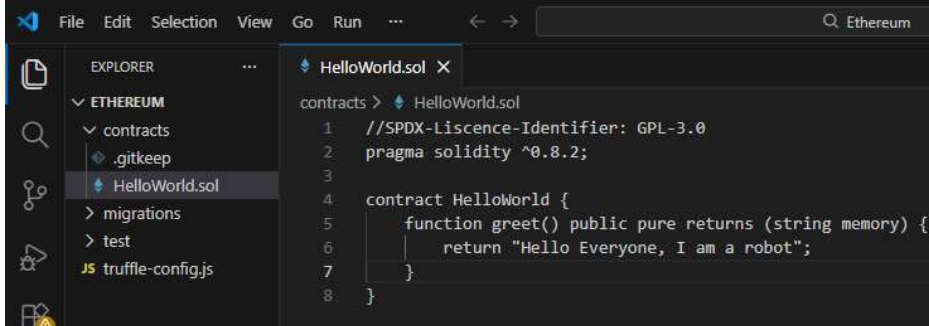
Init successful, sweet!

Try our scaffold commands to get started:
$ truffle create contract YourContractName # scaffold a contract
$ truffle create test YourTestName        # scaffold a test

http://trufflesuite.com/docs

PS C:\Users\Student\Desktop\Ethereum>
```

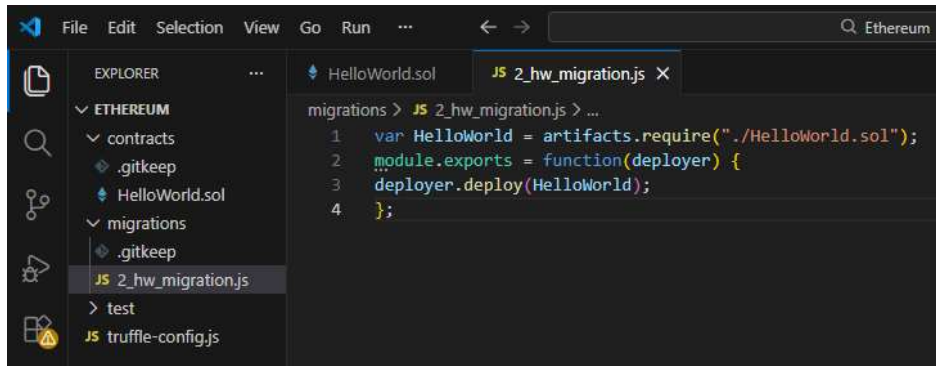
2. Creating the HelloWorld.sol contract inside the project.



The screenshot shows the VS Code interface with the Explorer pane on the left showing the 'contracts' folder containing 'HelloWorld.sol'. The Editor pane on the right shows the content of 'HelloWorld.sol':

```
contracts > HelloWorld.sol
1 //SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.8.2;
3
4 contract HelloWorld {
5     function greet() public pure returns (string memory) {
6         return "Hello Everyone, I am a robot";
7     }
8 }
```

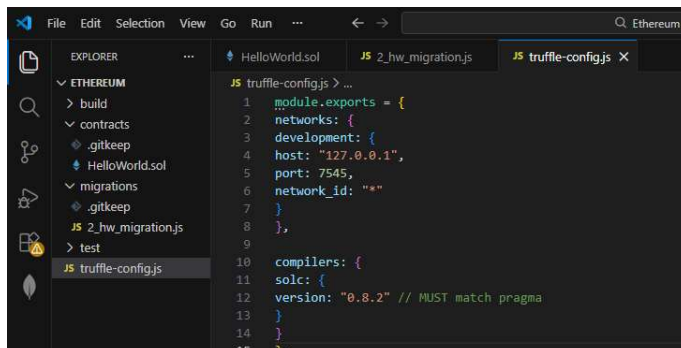
3. Created a new migration file with proper naming format.



The screenshot shows the VS Code interface with the Explorer pane on the left showing the 'migrations' folder containing '2_hw_migration.js'. The Editor pane on the right shows the content of '2_hw_migration.js':

```
migrations > JS 2_hw_migration.js > ...
1 var HelloWorld = artifacts.require("./HelloWorld.sol");
2 module.exports = function(deployer) {
3     deployer.deploy(HelloWorld);
4 };
```

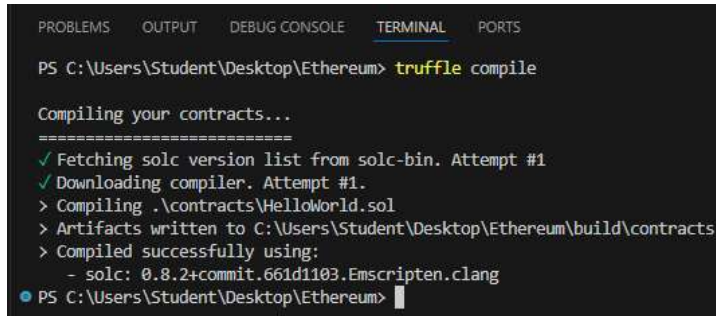
4. Setting up truffle config file



The screenshot shows the VS Code interface with the Explorer pane on the left showing the 'truffle-config.js' file. The Editor pane on the right shows the content of 'truffle-config.js':

```
JS truffle-config.js > ...
1 module.exports = {
2   networks: {
3     development: {
4       host: "127.0.0.1",
5       port: 7545,
6       network_id: "*"
7     },
8   },
9   compilers: {
10     solc: {
11       version: "0.8.2" // MUST match pragma
12     }
13   }
14 };
```

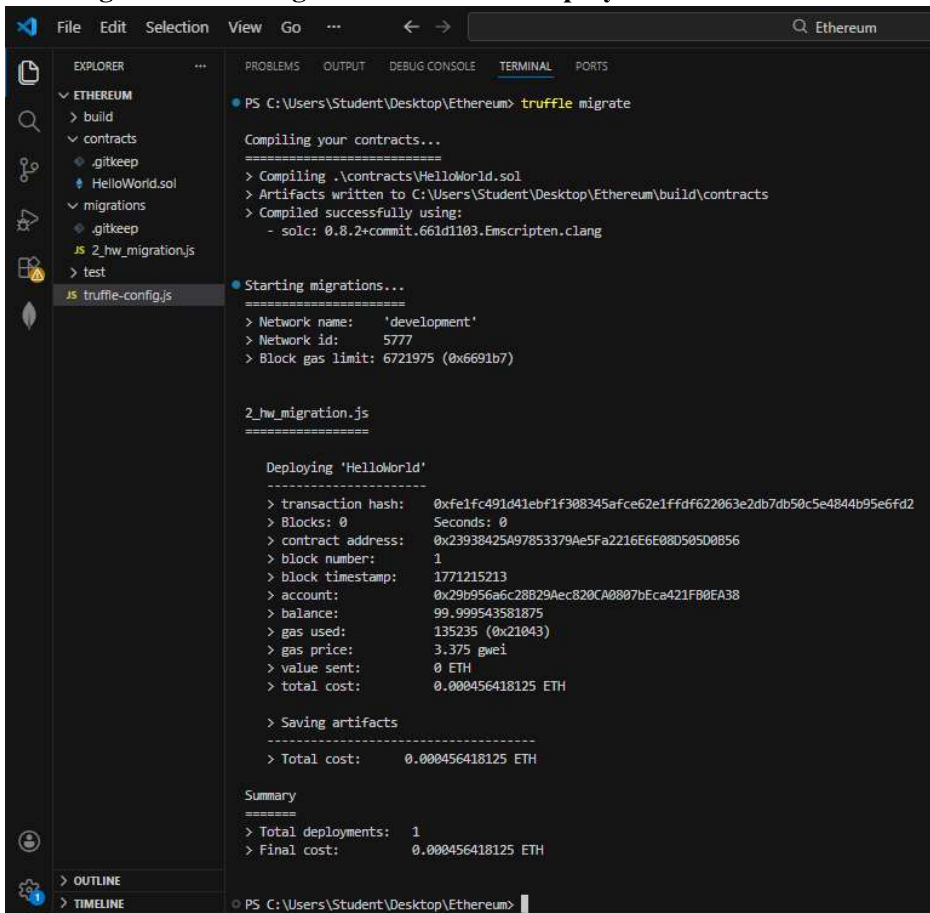
5. Truffle compile successful



```
PS C:\Users\Student\Desktop\Ethereum> truffle compile

Compiling your contracts...
=====
✓ Fetching solc version list from solc-bin. Attempt #1
✓ Downloading compiler. Attempt #1.
> Compiling .\contracts\HelloWorld.sol
> Artifacts written to C:\Users\Student\Desktop\Ethereum\build\contracts
> Compiled successfully using:
  - solc: 0.8.2+commit.661d1103.Emscripten.clang
PS C:\Users\Student\Desktop\Ethereum>
```

6. Running the truffle migrate command to deploy contract to blockchain network



```
PS C:\Users\Student\Desktop\Ethereum> truffle migrate

Compiling your contracts...
=====
> Compiling .\contracts\HelloWorld.sol
> Artifacts written to C:\Users\Student\Desktop\Ethereum\build\contracts
> Compiled successfully using:
  - solc: 0.8.2+commit.661d1103.Emscripten.clang

Starting migrations...
=====
> Network name: 'development'
> Network id: 5777
> Block gas limit: 6721975 (0x6691b7)

2_hw_migration.js
=====

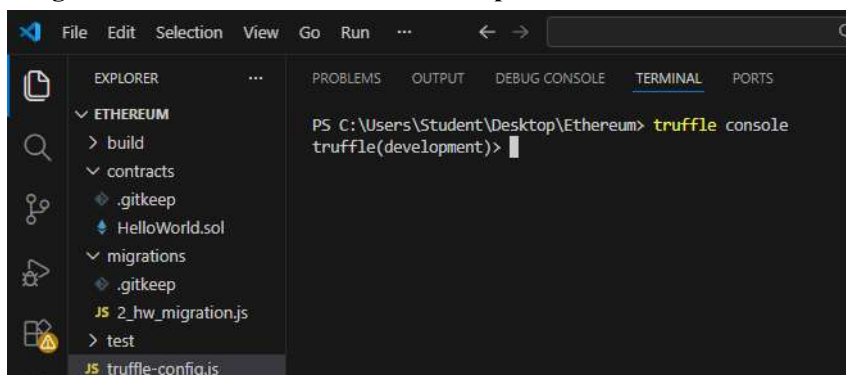
Deploying 'HelloWorld'
-----
> transaction hash: 0xfe1fc491d41ebf1f308345afce62e1ffdf622063e2db7db50c5e4844b95e6fd2
> Blocks: 0 Seconds: 0
> contract address: 0x23938425A97853379Ae5Fa2216E6E08D50500856
> block number: 1
> block timestamp: 1771215213
> account: 0x29b956a6c28829Aec820CA0807bEca421FB0EA38
> balance: 99.999543581875
> gas used: 135235 (0x21043)
> gas price: 3.375 gwei
> value sent: 0 ETH
> total cost: 0.000456418125 ETH

> Saving artifacts
-----
> Total cost: 0.000456418125 ETH

Summary
=====
> Total deployments: 1
> Final cost: 0.000456418125 ETH

PS C:\Users\Student\Desktop\Ethereum>
```

7. Using truffle console to start the development environment inside the terminal



```
PS C:\Users\Student\Desktop\Ethereum> truffle console
truffle(development)>
```


The screenshot shows the VS Code interface with the Explorer sidebar on the left and the Terminal panel on the right. The Explorer sidebar shows a project structure for 'ETHEREUM' with folders 'build', 'contracts', 'migrations', and 'test'. The 'contracts' folder is expanded, showing files like '.gitkeep', 'HelloWorld.sol', and 'truffle-config.js'. The 'test' folder is also expanded, showing '2_hw_migration.js'. The Terminal panel shows the output of a truffle console command:

```
PS C:\Users\Student\Desktop\Ethereum> truffle console
truffle(development)> let instance = await HelloWorld.deployed()
undefined
truffle(development)> instance.greet()
'Hello Everyone, I am Robot'
truffle(development)> |
```

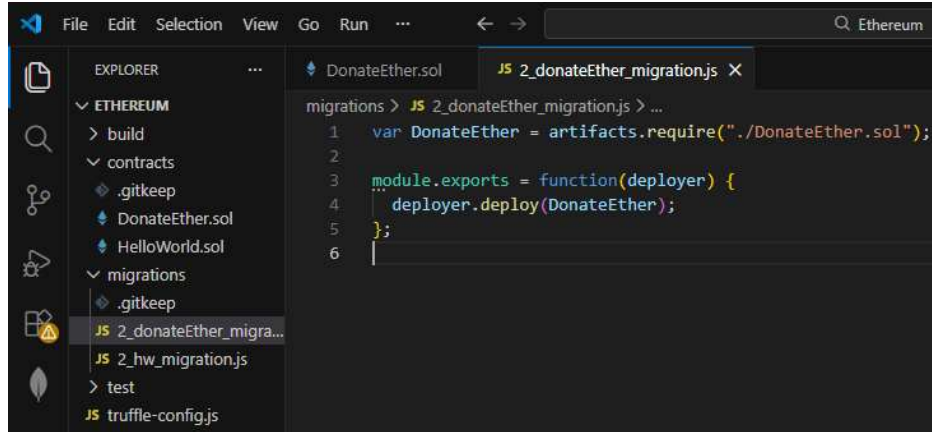
← BACK		BLOCK 1	
GAS USED	GAS LIMIT	MINED ON	BLOCK HASH
135235	6721975	2026-02-16 09:43:33	0xc91019cc477bde9566957be1baca4c8b166979cd45312dda06edd800918f7b1e
TX HASH 0xfef1fc491d41ebf1f308345afce62e1ffdf622063e2db7db50c5e4844b95e6fd2 CONTRACT CREATION			
FROM ADDRESS		CREATED CONTRACT ADDRESS	GAS USED
0x29b956a6c28b29Aec820CA807bEca421F80AE38		0x23938425A97853379Ae5Fa2216E6E08D505D0B56	135235
			VALUE
			0

← BACK TX 0xfe1fc491d41ebf1f308345afce62e1ffdf622063e2db7db50c5e4844b95e6fd2				
SENDER ADDRESS	0x29b956a6c28B29Aec80807bEca421FB0EA38	CREATED CONTRACT ADDRESS	0x23938425A97853379Ae5Fa2216E6E08D505D0B56	
		CONTRACT CREATION		
VALUE	GAS USED	GAS PRICE	GAS LIMIT	MINED IN BLOCK
0.00 ETH	135235	3375000000	169043	1
TX DATA				
0-608060405234801561001057600080fd5b5061017c8061026000396000f3fe08060405234801561001057600080fd5b50600431061002b5760003560e01c8063fae321714610030575b600080fd5b1003861004a5e5b6040516100845919061004c56b60405180910390f35b606004051806400160405280601a81526020017f4856c6c6f7245766572796fe652c20492061d620526f26f74080000000000815259095899056b6000100906826100e65506100a081856100f1565b93506100b0818560208601610102565b6100b98161013565b6040191505092915050565b60006020820190508181036008301520180de081846100b0565b995092915050565b60008151905091905056b6000601f19601f830110905091905056fea264697060f35821220f55ab58810840788ad5b080cb04e72e56cb739b2b9e8cfeb0b11b8174d95cf564736fc63430080020033				

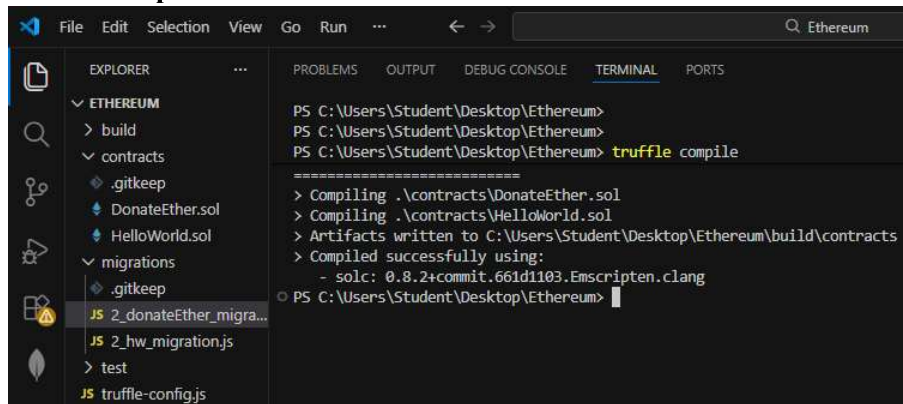
1. Write a program `DonateEther.sol`, create uint public variable 'balance' and initialize it to 0 using constructor() which is public, write a function `contribute()`, make it public and payable. Inside the function body write `balance+=msg.Value`; then create `migration.js` file for this contract as done in `HelloWorld` program. Deploy the contract on Ganache local Ethereum network using `truffle`.


```
contracts > DonateEther.sol
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract DonateEther {
5     uint public balance;
6
7     constructor() {
8         balance = 0;
9     }
10
11     function contribute() public payable {
12         balance += msg.value;
13     }
14
15     function getBalance() public view returns (uint) {
16         return balance;
17     }
18 }
```

Created a new migration file 2_donateEther_migration.js



Truffle compile



Migrating the new created contract using truffle migrate --reset

```
PS C:\Users\Student\Desktop\Ethereum> truffle migrate --reset

Compiling your contracts...
=====
> Compiling .\contracts\DonateEther.sol
> Compiling .\contracts\HelloWorld.sol
> Artifacts written to C:\Users\Student\Desktop\Ethereum\build\contracts
> Compiled successfully using:
- solc: 0.8.2+commit.661d1103.Emscripten.clang

Starting migrations...
=====
> Network name: 'development'
> Network id: 5777
> Block gas limit: 6721975 (0x6691b7)

2_donateEther_migration.js
=====

Deploying 'DonateEther'
=====
> transaction hash: 0x758114bcb5705acca81ed73ce4622f8475b6e1884339f0a8f7dac8371b761f54
> Blocks: 0
> contract address: 0x9E7A5ba7EDF36adae131793f083641130175d166
> block number: 2
> block timestamp: 1771216179
> account: 0x29b956a6c28829Aec820CA0807bEca421F80EA38
> balance: 99.998627754234138112
> gas used: 148637 (0x2449d)
> gas price: 3.270025889 gwei
> value sent: 0 ETH
> total cost: 0.000486046838063293 ETH

> Saving artifacts
=====
> Total cost: 0.000486046838063293 ETH
```

```
2_hw_migration.js
=====

Replacing 'HelloWorld'
=====
> transaction hash: 0xe6078269ecd2ea32b9cbbd3d21376267e80702f410c6c99757d114c9ebcc9aca
> Blocks: 0
> contract address: 0x62839d92C649b94c75bc893250188CDdcb229C37
> block number: 3
> block timestamp: 1771216179
> account: 0x29b956a6c28829Aec820CA0807bEca421F80EA38
> balance: 99.998627754234138112
> gas used: 135235 (0x21043)
> gas price: 3.178029377 gwei
> value sent: 0 ETH
> total cost: 0.000429780802798595 ETH

> Saving artifacts
=====
> Total cost: 0.000429780802798595 ETH

Summary
=====
> Total deployments: 2
> Final cost: 0.000915827640861888 ETH

PS C:\Users\Student\Desktop\Ethereum>
```

```
truffle(development)> let donate = await Donate[ether.deployed]()
undefined
truffle(development)> await donate.contribute({from: accounts[0], value: web3.utils.toWei("1", "ether")})
{
  tx: '0xe5b763b4ad2d9466381ad68db64d5d1eecea896bb7f5b2030e6916fdb0b3da08',
  receipt: {
    transactionHash: '0xe5b763b4ad2d9466381ad68db64d5d1eecea896bb7f5b2030e6916fdb0b3da08',
    transactionIndex: 0,
    blockNumber: 4,
    blockHash: '0xa4564e7c54b940ea608bd86a8c1df582ae53641014e4ba4fa7980dh394ae746',
    from: '0x29b956ac628b29aec820ca0807beca421fb0ea38',
    to: '0x9e7a5ba7edf36adae131793f08364113d175d166',
    cumulativeGasUsed: 43526,
    gasUsed: 43526,
    contractAddress: null,
    logs: [],
    logsBloom: '0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000'
  },
  status: true,
  effectiveGasPrice: 3096685913,
  type: 'tx2',
  rawLogs: []
},
logs: []
}
truffle(development)> await donate.getBalance()
BN {
  negative: 0,
  words: [ 56885248, 2993385, 222, <1 empty item> ],
  length: 3,
  red: null
}
```

← BACK		BLOCK 4	
GAS USED	GAS LIMIT	MINED ON	BLOCK HASH
43526	6721975	2026-02-16 10:01:48	0x44564e7c54b940ea608bd86a8c1dfe582a653641014e4ba4fa7980db394ae746
TX HASH		CONTRACT CALL	
0xe5b763b4ad2d9466301ad68db64d5d1eecea096bb7f5b2030e6916fbd0b3da08			
FROM ADDRESS	TO CONTRACT ADDRESS	GAS USED	VALUE
0x29b956a6c28b29Aec820CA0807bEca421FB0EA38	0x9E7A5ba7EDF36dae131793f083641130175d166	43526	100000000000000000

← BACK TX 0xe5b763b4ad2d9466301ad68db64d5d1eecea096bb7f5b2030e6916fbd0b3da08			
SENDER ADDRESS	TO CONTRACT ADDRESS		
0x29b956a6c28B29Aec820CA0807bEca421FB0EA38	0x9E7A5ba7EDF36adae131793f08364113D175d166	CONTRACT CALL	
VALUE	GAS USED	GAS PRICE	GAS LIMIT
1.00 ETH	43526	3096685913	54407
TX DATA		MINED IN BLOCK	
0xd7bb99ba		4	
EVENTS			

The screenshot shows the VS Code editor with the following details:

- Explorer Panel:**
 - ETHERIUM
 - build/contracts
 - DonateEther.json
 - employeeDatabase.json
 - HelloWorld.json
 - Storage.json
 - contracts
 - .gitkeep
 - DonateEther.sol
 - Employee.sol (selected)
 - helloworld.sol
 - storage.sol
 - migrations
 - .gitkeep
 - JS_2_deploy_donate.js
 - JS_2_new_migration.js
 - JS_3_deploy_employee.js
 - > test
 - JS_truffle-config.js
- Main Editor:**
 - File: Employee.sol
 - Line 1: // SPDX-License-Identifier: MIT
 - Line 2: pragma solidity ^0.8.0;
 - Line 4: contract employeeDatabase {
 - Line 5: struct Emp {
 - Line 6: uint empId;
 - Line 7: string empName;
 - Line 8: string empDept;
 - Line 9: string empRole;
 - Line 10: }
 - Line 12: Emp public employee;
 - Line 14: // Function to add employee details
 - Line 15: function addEmployee(uint _id, string memory _name, string memory _dept, string memory _role) public {
 - Line 16: employee = Emp(_id, _name, _dept, _role);
 - Line 17: }
 - Line 19: // Function to retrieve employee details
 - Line 20: function getEmployee() public view returns (uint, string memory, string memory, string memory) {
 - Line 21: return (employee.empId, employee.empName, employee.empDept, employee.empRole);
 - Line 22: }



```

C:\Users\Student\Documents\Ethereum>truffle compile

Compiling your contracts...
=====
> Compiling .\contracts\DonateEther.sol
> Compiling .\contracts\Employee.sol
> Compiling .\contracts\helloworld.sol
> Compiling .\contracts\storage.sol
> Artifacts written to C:\Users\Student\Documents\Ethereum\build\contracts
> Compiled successfully using:
   - solc: 0.8.2+commit.661d1103.Emscripten.clang

C:\Users\Student\Documents\Ethereum>truffle migrate --reset

Compiling your contracts...
=====
> Compiling .\contracts\DonateEther.sol
> Compiling .\contracts\Employee.sol
> Compiling .\contracts\helloworld.sol
> Compiling .\contracts\storage.sol
> Artifacts written to C:\Users\Student\Documents\Ethereum\build\contracts
> Compiled successfully using:
   - solc: 0.8.2+commit.661d1103.Emscripten.clang

```

```

3_deploy_employee.js
=====

Deploying 'employeeDatabase'
-----
> transaction hash: 0x8cf79e9fdf255dbc530f7cc80c6969d9553e70779aaa3c041e2133fc5e6e4949
> Blocks: 0          Seconds: 0
> contract address: 0xC1c2ff1684190C9bc7A6F56F5f364aAda84b146a
> block number: 8
> block timestamp: 1771218147
> account: 0x2acd44f58cbdb299FEC35c08C90007Ea205d7820
> balance: 93.996097266392467834
> gas used: 549281 (0x861a1)
> gas price: 2.854646356 gwei
> value sent: 0 ETH
> total cost: 0.001568003005070036 ETH

> Saving artifacts
-----
> Total cost: 0.001568003005070036 ETH

Summary
=====
> Total deployments: 3
> Final cost: 0.002360207896445916 ETH

```

```

C:\Users\Student\Documents\Ethereum>truffle console
truffle(development)> let instance = await employeeDatabase.deployed()
undefined
truffle(development)> await instance.addEmployee(101, "Styris", "IT", "Developer")
{
  tx: '0x0a8bea0d2a3dd114f9083bf7d589523e1d13e7211307af96a5b79836d4d8f5bd',
  receipt: {
    transactionHash: '0x0a8bea0d2a3dd114f9083bf7d589523e1d13e7211307af96a5b79836d4d8f5bd',
    transactionIndex: 0,
    blockNumber: 9,
    blockHash: '0x53c4666f20ca1b72e3f7933020c015c6f07e3d2c2c5eb5ea3434b29d4fe10557',
    from: '0x2acd44f58cbdb299fec35c08c90007ea205d7820',
    to: '0xc1c2ff1684190c9bc7a6f56f5f364aada84b146a',
    cumulativeGasUsed: 115039,
    gasUsed: 115039,
    contractAddress: null,

```


Date: 16 / 02 / 2026

Getting output to function calls correctly

Checking the blocks created for this purpose

[illegible]